

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA
UNIVERSITATEA DE STAT „ALECU RUSSO” DIN BĂLȚI
FACULTATEA DE ȘTIINȚE REALE, ECONOMICE ȘI ALE MEDIULUI
CATEDRA DE MATEMATICĂ ȘI INFORMATICĂ

**DEZVOLTAREA UNUI DASHBOARD INTERACTIV
UTILIZÂND REACT ȘI REACT-CHARTS PENTRU
ADMINISTRAREA UNUI TEREN SPORTIV**

TEZĂ DE AN

Autor:

Studentul grupei IS21Z

Oleg OALĂ

Conducător științific:

Adela GOREA

asist. univ.

BĂLȚI, 2025

CUPRINS

INTRODUCERE (Motivația alegerii temei, scopul și obiectivele lucrării)

1. ANALIZA DOMENIULUI ȘI SPECIFICAREA CERINȚELOR

- 1.1. Descrierea domeniului: necesitatea automatizării administrării unui stadion de mini-fotbal.
- 1.2. Analiza soluțiilor existente și argumentarea necesității unui sistem personalizat.
- 1.3. Specificarea cerințelor funcționale (gestionare echipe, rezervări ore, gestiune financiară, angajați) și non-funcționale.

2. FUNDAMENTAREA TEORETICĂ ȘI TEHNOLOGIILE UTILIZATE

- 2.1. Arhitectura sistemului (Client-Server, API RESTful).
- 2.2. Tehnologii pentru frontend: React.js și React-Charts pentru interfața interactivă. (Aici putem refolosi teoria din teza ta de an)
- 2.3. Tehnologii pentru backend: Node.js cu framework-ul Express.js (recomandat, ca să folosim tot JavaScript/TypeScript).
- 2.4. Sisteme de Gestiune a Bazelor de Date: alegerea între SQL (PostgreSQL) sau NoSQL (MongoDB).

3. PROIECTAREA ȘI ARHITECTURA SISTEMULUI

- 3.1. Arhitectura aplicației și schemele de comunicare frontend-backend.
- 3.2. Proiectarea bazei de date (Schemele pentru Entități: Utilizatori/Angajați, Echipe, Rezervări/Programări, Finanțe - încasări, cheltuieli, datorii).
- 3.3. Proiectarea interfețelor utilizator (Wireframes/Design pentru Dashboard-ul administratorului).

4. IMPLEMENTAREA PRACTICĂ A PLATFORMEI

- 4.1. Dezvoltarea serverului backend, autentificarea (JWT) și gestionarea angajaților.
- 4.2. Construirea interfeței web (frontend) pe baza proiectului anterior.
- 4.3. Implementarea modulelor principale:

Modulul de Rezervări (calendarul orelor pentru echipe).

Modulul Financiar (calculul încasărilor, evidența cheltuielilor, casa și datoriile, afișate prin React-Charts).

4.4. Integrarea datelor dinamice în graficele interactive.

5. TESTAREA SISTEMULUI ȘI MANUALUL DE UTILIZARE

5.1. Testarea funcționalităților (backend și frontend).

5.2. Ghid de utilizare a platformei pentru administratorul stadionului.

CONCLUZII

BIBLIOGRAFIE

ANEXE (Fragmente de cod important, structura bazei de date)

INTRODUCERE

În contextul creșterii accelerate a necesității de digitalizare și a cerințelor tot mai ridicate de optimizare a afacerilor, dashboard-urile interactive și sistemele de management au devenit o componentă esențială în infrastructura organizațiilor moderne. Ele facilitează accesul rapid la informații relevante, permit analiza fluxurilor financiare și oferă utilizatorilor posibilitatea de a interacționa cu datele într-un mod vizual și intuitiv.

Importanța acestor sisteme integrate este reflectată în administrarea eficientă a afacerilor sportive:

- În managementul timpului, acestea sunt utilizate pentru a monitoriza rezervările orelor de joc, evitând suprapunerile și optimizând orarul echipelor;
- În gestiunea financiară, dashboard-urile ajută la prezentarea încasărilor, a cheltuielilor și a datoriilor în mod clar și concis, facilitând evidența banilor din casă;
- În managementul administrativ, acestea sunt folosite pentru transparență, planificarea resurselor și luarea deciziilor strategice bazate pe date.

Lucrarea de față își propune dezvoltarea unui dashboard interactiv complex (aplicație Full-Stack) utilizând tehnologii moderne și complementare:

- React – o bibliotecă JavaScript creată de Facebook, recunoscută pentru viteza de dezvoltare și arhitectura component-based, utilizată pentru interfața frontend;
- React-Charts – o bibliotecă open-source pentru vizualizarea datelor, ce oferă suport pentru grafice interactive și responsive;
- Node.js și Baze de Date (backend) – tehnologii utilizate pentru stocarea securizată a datelor despre angajați, echipe, rezervări și tranzacții financiare.

Prin îmbinarea acestor tehnologii, se urmărește obținerea unei aplicații web care să fie: ușor de utilizat, cu o interfață prietenoasă; scalabilă, pentru a putea fi extinsă în funcție de nevoi (adăugarea de noi funcționalități pe viitor); performantă, prin optimizarea fluxului de date și a interacțiunilor în timp real.

Scopul lucrării: Cercetarea funcționalităților bibliotecilor React și React-Charts și realizarea unui dashboard interactiv ce permite gestionarea programărilor și vizualizarea datelor în mod dinamic.

Pentru atingerea scopului au fost formulate următoarele obiective specifice:

Trebuie concluzie la sfarsit la fiecare tema

- Analiza principiilor de funcționare a bibliotecilor React și React-Charts;
- Analiza principiilor de funcționare a framework-ului Node.js;
- Analiza cerințelor de administrare a unui stadion de mini;
- Proiectarea bazei de date și a arhitecturii aplicației client-server;
- Realizarea părții backend pentru gestionarea angajaților, a echipelor și a operațiunilor financiare;
- Integrarea interfeței frontend (React și React-Charts) cu baza de date;
- Testarea interfeței și a funcționalităților sistemului.

Lucrarea este structurată în introducere, patru capitole și o listă bibliografică.

- Capitolul 1: ANALIZA DOMENIULUI ȘI SPECIFICAREA CERINȚELOR – descrie problema administrării stadionului și cerințele aplicației;
- Capitolul 2: FUNDAMENTAREA TEORETICĂ ȘI TEHNOLOGIILE UTILIZATE – oferă o fundamentare asupra arhitecturii web și a tehnologiilor React și Node.js;
- Capitolul 3: PROIECTAREA ȘI ARHITECTURA SISTEMULUI – prezintă arhitectura aplicației și structura bazei de date;
- Capitolul 4: IMPLEMENTAREA PRACTICĂ A PLATFORMEI – prezintă dezvoltarea codului, modulele de gestionare a echipelor și a finanțelor.

Lucrarea este expusă pe ... pagini, conține ... figuri și ... referințe bibliografice.

1. ANALIZA DOMENIULUI ȘI SPECIFICAREA CERINȚELOR

Capitolul de față are drept scop fundamentarea necesității de elaborare a o aplicației bazată pe web printr-o detaliere analitică în legătură cu domeniul de referință respectiv, adică administrarea unui teren sportiv. Un produs software pentru a fi util și viabil trebuie să rezolve câteva probleme reale din activitatea zilnică a utilizatorului. În continuare în secțiunile următoare va fi o descriere despre procesele de gestionare legate de un stadion de mini-fotbal, vor fi identificate deficiențele existente în metodele tradiționale pentru dovezi și vor fi extrase Cerințe funcționale și non-funcționale care sunt necesare pentru dashboard-ul ce le va îndeplini.

1.1. Descrierea domeniului: necesitatea automatizării administrării unui stadion de mini-fotbal

Managementul modern al unei facilități sportive, precum un stadion de mini-fotbal, este o activitate antreprenorială complexă care depășește simpla punere la dispoziție a unui teren de joc. În esență, acest domeniu reprezintă o intersecție constantă între gestionarea timpului, coordonarea resurselor umane (angajați și clienți) și controlul riguros al fluxurilor financiare. Pentru un administrator, provocarea principală nu constă în organizarea jocurilor propriu-zise, ci în menținerea unei evidențe clare a tuturor variabilelor de business care asigură rentabilitatea locației.

O analiză aprofundată a proceselor operaționale dintr-un astfel de complex sportiv evidențiază mai multe direcții critice de activitate. În primul rând, managementul rezervărilor și al timpului reprezintă nucleul afacerii. Terenul este o resursă fixă, închiriată pe intervale orare (de regulă, cu ora). Evidența manuală a acestor programări, realizată frecvent prin agende clasice sau aplicații de mesagerie, este inefficientă și predispusă la erori umane. Apar frecvent situații de dublă rezervare a aceluiași interval, anulări neînregistrate la timp sau dificultăți în a identifica rapid „ferestrele” libere pentru clienții noi.

În al doilea rând, componenta financiară necesită o monitorizare zilnică și extrem de precisă. Activitatea stadionului generează un flux de numerar dinamic. Administratorul trebuie să țină o evidență clară a încasărilor și, la fel de important, a cheltuielilor operaționale. Întreținerea bazei sportive presupune costuri diverse: plata utilităților mentenanța gazonului sintetic, reparații curente ale infrastructurii, achiziția de consumabile și salarizarea personalului. Fără un sistem centralizat, calculul manual al balanței dintre încasări și cheltuieli pentru a afla profitul real – adică banii din casă la finalul zilei sau al lunii – devine o sarcină administrativă obositoare și mare consumatoare de timp.

O specificitate majoră a acestui domeniu, care justifică din plin necesitatea unui sistem informatic personalizat, este gestionarea echipelor și a plăților amânate. În practica comercială a

stadionelor de mini-fotbal, o mare parte dintre clienți sunt echipe constante, formate din jucători amatori care rezervă terenul săptămânal. Datorită acestei fidelități, se practică frecvent sistemul de plată periodică (la finalul lunii) sau pot apărea restanțe punctuale. Astfel, generarea unei secțiuni pentru datorii este absolut vitală. Evidența acestor creanțe pe suport de hârtie duce inevitabil la confuzii, pierderi financiare și chiar neînțelegeri cu clienții. Administratorul are nevoie de o vizualizare imediată a sumelor spre achitare, per echipă, pentru a ști exact cine și cât datorează.

Metodele tradiționale de gestionare sunt limitate, fragmentate și nu oferă o perspectivă analitică. Utilizarea instrumentelor nespecializate nu permite generarea de rapoarte automate și nu oferă o interfață vizuală prin care datele să poată fi interpretate rapid. Prin urmare, automatizarea acestor procese de rutină prin intermediul unui Dashboard inteligent devine nu doar un avantaj competitiv, ci o necesitate.

Trecerea la o soluție digitală îi permite beneficiarului să delege calculele matematice și evidența stocării către server. Un astfel de sistem consolidează toate informațiile – echipe, ore jucate, încasări, cheltuieli, solduri de casă și datorii – într-o singură bază de date securizată, prezentându-le pe frontend sub forma unor grafice interactive. În acest fel, timpul alocat birocrăției este redus drastic, lăsând loc pentru decizii manageriale eficiente și o mai bună organizare a activității practice a stadionului.

1.2. Analiza soluțiilor existente și argumentarea necesității unui sistem personalizat

Înainte de a pune bazele arhitecturale ale oricărui sistem informatic, ne simțim obligați să scanăm riguros soluțiile care rulează deja pe piață. Pentru gestiunea spațiilor de mini-fotbal, de exemplu, opțiunile variază agresiv, de la simple unelte de uz general la platforme comerciale rigide. Dar aici apare discrepanța. O implementare de tip „off-the-shelf” nu reprezintă deloc o garanție pentru rezolvarea blocajelor specifice cu care se luptă un administrator în viața reală. Practic, am observat că aceste soluții prefabricate forțează de cele mai multe ori utilizatorul să își schimbe radical fluxul de lucru pentru a se plia pe limitele software-ului, și nu invers.

Plecând de la constrângerile tehnice identificate în teren, am ales să ne concentrăm pe o abordare software personalizată. E singura cale logică. Majoritatea administratorilor de baze sportive se bazează încă pe instrumente rudimentare, de tip foaie de calcul, mizând pe simplitatea Excel-ului sau a variantelor cloud din suita Google. Într-o primă fază, par soluții ideale. Însă, din perspectiva unei structuri de date relaționale, observăm rapid că acest model „plat” eșuează lamentabil în a gestiona o logică de afaceri complexă. Practic, un astfel de document nu poate impune constrângeri (constraints) de integritate. Nimic nu împiedică sistemul să valideze două rezervări pe același teren la aceeași oră, în afara vigilenței umane. Aici apare haosul. Urmărirea datoriilor și a încasărilor amânate, pe măsură ce volumul de informație crește, devine o sarcină aproape imposibilă, erorile de

calcul fiind, la drept vorbind, inevitabile. Securitatea rămâne o simplă iluzie; integritatea formulelor depinde exclusiv de faptul că utilizatorul nu apasă din greșeală tasta greșită.

Ne-am uitat, desigur, și spre soluțiile de tip ERP sau CRM deja existente pe piață. Sunt sisteme solide, n-avem ce zice. Totuși, pentru specificul administrării unui stadion, aplicarea lor este contraproductivă. Ne lovim de o redundanță tehnologică masivă, unde modulele de gestiune a stocurilor complexe sau fluxurile de marketing aglomerează inutil interfața de lucru. Procesul de învățare se prelungește nepermis de mult din cauza unei arhitecturi supradimensionate. Mai mult, licențele costisitoare și abonamentele lunare specifice modelului SaaS nu fac decât să destabilizeze un buget de operare limitat. Resursele sunt irosite pe funcționalități care nu vor fi niciodată apelate în fluxul zilnic de rezervări.

Există, de asemenea, platforme web dedicate exclusiv rezervărilor pentru terenuri sportive. Deși acestea rezolvă eficient problema calendarului și a vizibilității orelor disponibile, ele prezintă un dezavantaj major din perspectiva managementului intern: lipsa unui modul financiar detaliat. Aceste aplicații sunt orientate preponderent spre clientul final (jucătorul) și mai puțin spre administrator. Ele nu permit o evidență granulară a „banilor din casă”, nu integrează cheltuielile operaționale de mentenanță (iluminat, salarizare, reparații) și nu sunt adaptate pentru modelul de afaceri local în care echipele fidele pot acumula datorii care trebuie achitate la sfârșitul lunii.

Având în vedere aceste deficiențe majore ale soluțiilor existente, dezvoltarea unui sistem informatic personalizat devine o necesitate strategică. Elaborarea unui Dashboard propriu, structurat exact pe fluxul de lucru real al beneficiarului, elimină funcționalitățile inutile și se concentrează exclusiv pe modulele critice: gestionarea echipelor, a programărilor, a încasărilor, a cheltuielilor și a datoriilor.

O arhitectură sistematizată creată special a fost dezvoltată prin integrarea unei interfețe interactive legate de React și un backend solid cu o bază de date proprie ce aparține Nodejs care oferă flexibilitate totală. Interfața este proiectată ergonomic astfel încât să asigure o înțelegere rapidă a datelor și transformarea acestora în grafice vizuale prin ReactCharts oferind persoanei responsabile O imagine clară despre profitabilitate și datoriile recuperabile. Din punct de vedere tehnic și economic această aplicație oferă independență totală eliminând taxele de subscripție și asigurând control complet asupra securității și stocării datelor financiare legate de stadion.

1.3. Specificarea cerințelor funcționale și non-funcționale

În ingineria sistemelor software, etapa de specificare a cerințelor reprezintă fundamentul pe care se va construi întreaga arhitectură a aplicației. O definiție clară și riguroasă a acestor cerințe asigură că produsul final va corespunde exact nevoilor beneficiarului. Pentru dezvoltarea platformei destinate administrării stadionului de mini-fotbal, cerințele au fost extrase în urma analizei fluxurilor de lucru zilnice și au fost divizate în două categorii majore: cerințe funcționale (care descriu acțiunile pe care sistemul trebuie să le execute) și cerințe non-funcționale (care definesc atributele de calitate, performanță și securitate ale sistemului).

Cerințe funcționale Cerințele funcționale dictează comportamentul aplicației și modulele pe care aceasta trebuie să le conțină pe partea de frontend și backend. Pentru a asigura un management eficient al stadionului, sistemul trebuie să îndeplinească următoarele sarcini esențiale:

- Modulul de Autentificare și Securitate: Sistemul trebuie să permită accesul doar utilizatorilor autorizați (administratorului) pe baza unui nume de utilizator și a unei parole criptate, blocând accesul public la datele financiare.
- Modulul de Gestiune a Echipelor: Sistemul trebuie să permită adăugarea, editarea și ștergerea profilelor echipelor de fotbal care închiriază terenul. Fiecare echipă va avea asociat un istoric al rezervărilor și un profil financiar.
- Modulul de Programări (Managementul timpului): Aplicația trebuie să ofere o interfață interactivă (calendar sau tabel orar) prin care administratorul să poată alocă anumite ore de joc pentru fiecare echipă. Sistemul trebuie să calculeze automat numărul total de ore jucate de o echipă într-o anumită perioadă (săptămână/lună).
- Modulul Financiar (Încasări și Cheltuieli): Aplicația trebuie să permită introducerea sumelor încasate de la clienți, precum și înregistrarea cheltuielilor operaționale ale stadionului (facturi de energie electrică pentru iluminat, salarizarea personalului, costuri de mentenanță și curățenie).
- Modulul de evidență a Datoriilor: Sistemul va calcula automat balanța financiară a fiecărei echipe. Dacă o echipă a jucat un număr de ore, dar nu a achitat suma integrală, aplicația va muta diferența într-o secțiune dedicată datoriilor („restanțe spre achitare”), oferind o evidență clară a banilor ce trebuie recuperați.
- Dashboard-ul analitic (Generarea Rapoartelor): Interfața principală va centraliza toate datele financiare, calculând automat „banii din casă” (profitul net obținut din diferența

dintre încasări și cheltuieli). Aceste date vor fi vizualizate prin intermediul unor grafice interactive (React-Charts) pentru o analiză statistică rapidă.

- Cerințe non-funcționale Aceste cerințe asigură stabilitatea, performanța și calitatea experienței de utilizare a platformei, aspecte vitale pentru un instrument de lucru zilnic:
- Performanță și timp de răspuns: Deoarece platforma este construită ca o aplicație SPA (Single Page Application) cu React, timpul de încărcare a paginilor și de redare a graficelor trebuie să fie minim (sub 2 secunde), iar actualizarea datelor în panoul de control trebuie să se realizeze instantaneu.
- Ergonomie și interfață intuitivă (Usability): Interfața utilizatorului (UI) trebuie să fie clară și lipsită de elemente redundante, punând accent pe utilitate. Administratorul trebuie să poată introduce o plată sau o rezervare cu un număr minim de click-uri.
- Responsivitate (Design adaptabil): Platforma trebuie să fie complet funcțională și vizual optimizată pentru orice dispozitiv. Deși sistemul este gestionat preponderent de pe un laptop, el trebuie să fie accesibil și adaptat vizual pentru ecranele telefoanelor mobile, permițând administratorului să verifice situația stadionului de oriunde.
- Portabilitate (Cross-platform): Fiind o aplicație web, sistemul nu va necesita instalare și va rula direct din browser, fiind pe deplin compatibil cu orice sistem de operare (macOS, Windows, Linux).
- Securitatea și stocarea datelor: Baza de date (backend-ul) trebuie să asigure integritatea datelor introduse. Structura bazei de date trebuie proiectată astfel încât să prevină pierderea informațiilor financiare în cazul unor erori de rețea.

Implementarea cu strictețe a acestor cerințe va garanta crearea unui produs software viabil, transformând teza de licență dintr-un simplu proiect teoretic într-un instrument de business real și eficient.

2. FUNDAMENTAREA TEORETICĂ ȘI TEHNOLOGIILE UTILIZATE

Odată cu stabilirea cerințelor funcționale și non-funcționale în capitolul anterior, este necesară identificarea și justificarea stivei tehnologice care va fi utilizată pentru dezvoltarea produsului software. Trecerea de la un simplu prototip vizual la o aplicație complexă de tip Full-Stack presupune îmbinarea mai multor tehnologii complementare. Prezentul capitol detaliază fundamentul teoretic al arhitecturii Client-Server, tehnologiile moderne utilizate pentru dezvoltarea interfeței grafice interactive (frontend), precum și platformele selectate pentru gestionarea logicii de business și stocarea securizată a datelor (backend).

2.1. Arhitectura sistemului Client-Server și utilizarea API-urilor RESTful

Pentru a dezvolta un Dashboard inteligent capabil să gestioneze fluxuri continue de date – precum înregistrarea echipelor, evidența rezervărilor, stocarea încasărilor și calculul automat al datoriilor – este imperativă adoptarea unei arhitecturi software decuplate. În ingineria modernă a sistemelor web, arhitectura tradițională monolitică (unde codul vizual și cel de bază de date sunt amestecate) a fost înlocuită de modelul **Client-Server**. Această paradigmă reprezintă fundamentul pe care este construită aplicația destinată administrării stadionului de mini-fotbal.

În cadrul arhitecturii Client-Server, sistemul este divizat în două entități distincte, dar interdependente, care comunică permanent prin intermediul rețelei. **Cientul (frontend-ul)** reprezintă nivelul de prezentare. Acesta este executat direct în browserul web al utilizatorului (administratorul complexului sportiv). Rolul clientului este de a afișa interfața grafică (tabele, butoane, grafice financiare) și de a capta interacțiunile utilizatorului. Cu toate acestea, din motive de securitate și performanță, clientul nu are permisiunea și nici capacitatea de a stoca date sensibile pe termen lung sau de a executa calcule financiare complexe în mod autonom.

Aici intervine **Serverul (backend-ul)**, care funcționează ca nucleul central al aplicației. Serverul este găzduit pe o mașină de calcul la distanță (sau în cloud) și este responsabil pentru logica de business. Când administratorul introduce o nouă plată în sistem, backend-ul preia informația, o validează, actualizează balanța financiară a echipei respective (scăzând datoria) și înregistrează tranzacția într-o **Bază de Date**. De asemenea, serverul gestionează mecanismele de autentificare, asigurându-se că informațiile confidențiale, precum „banii din casă”, nu sunt accesibile persoanelor neautorizate.

Comunicarea eficientă între aceste două straturi arhitecturale se realizează prin intermediul unui **API (Application Programming Interface)**, mai exact un API de tip **RESTful**

(Representational State Transfer). REST este un set de reguli și standarde care utilizează protocolul de rețea HTTP pentru a facilita schimbul de date asincron. În contextul acestui Dashboard, ori de câte ori interfața grafică are nevoie de date noi (de exemplu, pentru a desena graficul cu încasările pe ultima lună), frontend-ul expediază o cerere (Request) de tip GET către un anumit capăt de rețea al serverului (endpoint). Serverul primește cererea, interoghează baza de date, extrage sumele necesare, le împachetează într-un format text standardizat, ușor de procesat – denumit **JSON** (JavaScript Object Notation) – și le trimite înapoi către Client (Response). Similar, la adăugarea unei noi echipe în sistem, se utilizează o cerere de tip POST, care poartă cu sine datele ce trebuie salvate.

Implementarea acestei arhitecturi separă clar responsabilitățile (principiul *Separation of Concerns*). Primul avantaj tehnic major este **scalabilitatea și flexibilitatea**. Dacă în viitor administrarea stadionului va necesita o aplicație mobilă dedicată pentru jucători (pentru a-și vedea propriile datorii), serverul și baza de date vor rămâne intacte; va fi necesară exclusiv dezvoltarea noii interfețe mobile, care se va conecta la același API RESTful. Al doilea avantaj critic este **securitatea datelor**: structura bazei de date și algoritmi de calcul rămân complet ascunși pe server, feriți de manipulări externe.

Astfel, adoptarea arhitecturii Client-Server decuplate, mediată de un API RESTful asincron, oferă cadrul optim pentru un Dashboard rapid, securizat și extensibil, capabil să răspundă nevoilor complexe de management administrativ și financiar.

2.2. Tehnologii pentru frontend: React.js și React-Charts pentru interfața interactivă

În arhitectura platformei dezvoltate pentru administrarea stadionului de mini-fotbal, nivelul de prezentare (frontend) reprezintă punctul unic de interacțiune dintre administrator și sistemul informatic. O interfață de tip Dashboard are cerințe stricte în ceea ce privește performanța, claritatea vizuală și capacitatea de a actualiza informațiile instantaneu. Pentru a satisface aceste exigențe, dezvoltarea părții de client s-a realizat utilizând **React.js**, una dintre cele mai populare și robuste biblioteci JavaScript, completată de **React-Charts** pentru vizualizarea analitică a datelor financiare.

React.js și Arhitectura Bazată pe Componente Elementul central care diferențiază React de abordările tradiționale de dezvoltare web este arhitectura sa bazată pe componente (Component-Based Architecture). Această paradigmă presupune divizarea interfeței utilizatorului în elemente logice mici, independente și reutilizabile. În contextul acestui proiect, în loc de a scrie un document HTML masiv și greu de întreținut, interfața a fost segmentată modular. De exemplu, meniul lateral de navigare (Sidebar), tabelul care afișează istoricul echipelor, formularul pentru introducerea unei

noi rezervări de ore și cardurile de sinteză financiară (care indică sumele din casă) reprezintă, fiecare în parte, componente React distincte.

Această modularitate nu doar că simplifică procesul de scriere și depanare a codului, dar permite și o scalabilitate ridicată a aplicației. Dacă, pe viitor, se dorește adăugarea unui modul nou (de exemplu, un program de loialitate pentru echipele fidele), acesta poate fi integrat ca o componentă nouă, fără a afecta funcționalitatea modulelor deja existente.

Un alt concept teoretic fundamental al bibliotecii React este utilizarea unui **Virtual DOM** (Document Object Model virtual). În aplicațiile web clasice, orice modificare a datelor (de exemplu, ștergerea unei echipe sau înregistrarea unei plăți) necesită recalcularea și reîncărcarea întregii pagini în browser, un proces lent și inefficient. React soluționează această problemă prin menținerea unei copii „virtuale” a interfeței în memoria memorie. Când administratorul introduce o nouă plată pentru a reduce datoria unei echipe, React compară rapid noul Virtual DOM cu versiunea anterioară și actualizează în browser exclusiv acea mică porțiune de ecran care s-a modificat (tabelul financiar). Acest mecanism garantează o fluiditate excepțională și un timp de răspuns instantaneu, esențiale pentru un instrument operațional utilizat zilnic.

Gestionarea Stării cu React Hooks Pentru a manipula fluxul de date dinamice și asincrone primit de la server, dezvoltarea modernă în React utilizează mecanismul de „Hooks”. Printre cele mai importante funcții implementate în acest Dashboard se numără `useState` și `useEffect`. Funcția `useState` este utilizată pentru a stoca temporar datele la nivel de interfață (cum ar fi textul introdus în câmpul de căutare a unei anumite echipe), în timp ce `useEffect` gestionează efectele secundare. Rolul crucial al hook-ului `useEffect` este de a iniția cererile HTTP (request-urile) către backend pentru a extrage listele actualizate cu rezervări și datorii imediat ce pagina Dashboard-ului este accesată.

React-Charts și Vizualizarea Datelor O funcționalitate definitorie a oricărui Dashboard analitic este capacitatea de a transforma seturile de date brute în informații vizuale ușor de interpretat. Administrarea eficientă a stadionului necesită o analiză rapidă a balanței dintre cheltuielile operaționale și încasările realizate. În acest scop, aplicația integrează biblioteca **React-Charts**.

React-Charts este un instrument open-source specializat în redarea graficelor interactive, perfect compatibil cu arhitectura React. După ce interfața preia datele din baza de date sub formă de obiecte JSON (conținând istoricul plăților și al cheltuielilor), acestea sunt procesate și introduse în componentele React-Charts. Rezultatul constă în generarea unor diagrame dinamice (grafice cu bare sau diagrame liniare) care ilustrează vizual performanța afacerii. Aceste reprezentări îi permit

administratorului să identifice, într-o fracțiune de secundă, perioadele de vârf ale închirierilor, ponderea datoriilor în raport cu încasările totale și nivelul real al banilor din casă, transformând platforma într-un instrument indispensabil pentru luarea deciziilor.

2.3. Tehnologii pentru backend și gestionarea datelor: Node.js și Baze de Date

Trecerea de la o simplă interfață vizuală la un sistem de management complet (Full-Stack) presupune implementarea unei infrastructuri de server capabile să prelucreze, să securizeze și să stocheze permanent informațiile. În cazul administrării stadionului de mini-fotbal, o pană de curent sau închiderea browser-ului nu trebuie să ducă la pierderea istoricului echipelor, a programărilor sau a balanței financiare (banii din casă și datoriile). Pentru a garanta persistența și integritatea acestor date, arhitectura aplicației necesită un mediu de execuție pe server (backend) și un Sistem de Gestionare a Bazelor de Date (SGBD).

Node.js și avantajul ecosistemului unificat Din punct de vedere istoric, dezvoltarea serverelor web impunea utilizarea unor limbaje de programare distincte de cele folosite pentru interfață (precum PHP, Java sau C#). Apariția **Node.js** a revoluționat acest proces, permițând executarea codului JavaScript direct pe server, în afara browser-ului. Pentru dezvoltarea acestui Dashboard, alegerea Node.js reprezintă o decizie strategică majoră. Aceasta permite utilizarea aceluiași limbaj de programare (JavaScript/TypeScript) atât pe frontend (prin React), cât și pe backend. Acest „ecosistem unificat” simplifică dezvoltarea, reduce timpul de implementare și facilitează transferul eficient al pachetelor de date de tip JSON între client și server.

Arhitectura internă a Node.js este bazată pe un model asincron, condus de evenimente (event-driven) și non-blocant (non-blocking I/O). În contextul aplicației noastre, acest lucru înseamnă că serverul poate procesa solicitări multiple simultan. Dacă administratorul introduce o nouă încasare, în timp ce sistemul generează automat un raport cu datoriile cumulate ale echipelor pe luna în curs, serverul nu se va bloca așteptând finalizarea primei sarcini, ci le va procesa eficient în paralel, asigurând un timp de răspuns optim.

Express.js pentru construcția API-ului RESTful Pentru a structura comunicarea pe server, Node.js este acompaniat de **Express.js**, cel mai popular framework minimal pentru aplicații web. Express facilitează crearea „rutelor” (endpoints) – adresele de rețea specifice la care frontend-ul face cereri. De exemplu, platforma utilizează rute distincte pentru entități diferite: o cerere de tip GET /api/teams va extrage lista tuturor echipelor de fotbal, în timp ce o cerere POST /api/payments va înregistra o nouă plată și va actualiza automat balanța banilor din casă. Express preia aceste cereri, execută

logica de business (calculule matematice pentru profit și datorii) și comunică mai departe cu baza de date.

Sisteme de Gestiune a Bazelor de Date (SGBD) Nucleul central al persistenței informaționale este baza de date. În dezvoltarea aplicațiilor web moderne, există două abordări principale: bazele de date relaționale (SQL, precum PostgreSQL sau MySQL) și cele non-relaționale (NoSQL, precum MongoDB).

Pentru gestiunea unui complex sportiv, datele au o structură foarte bine definită și interconectată. O anumită plată financiară trebuie să fie legată strict de o anumită „Echipă” și de o anumită „Rezervare” orară. Prin urmare, un model de date structurat este vital pentru a menține integritatea financiară și a evita discrepanțele. Baza de date stochează scheme precise: profilul fiecărei echipe, tariful pe oră, orele jucate, sumele achitate și restanțele. Ori de câte ori se realizează o modificare (ex: o echipă își achită datoria), serverul Node.js interoghează baza de date, modifică valorile și returnează noile rezultate către Dashboard pentru a actualiza graficele.

Securitate și Autentificare (JWT) Nu în ultimul rând, tehnologia de backend este responsabilă pentru securitatea sistemului. Fiind un Dashboard care gestionează încasări și calculează profitul net, accesul public trebuie strict interzis. Sistemul utilizează mecanismul **JWT (JSON Web Token)** pentru autentificare. Când administratorul introduce datele corecte de logare, serverul generează un „token” criptat unic, valabil pentru o perioadă determinată. Orice acțiune ulterioară în aplicație – fie că este vorba de vizualizarea calendarului sau de ștergerea unei rezervări – necesită prezența acestui token în cererea HTTP. Astfel, se garantează că nicio persoană neautorizată nu poate manipula sau extrage datele financiare ale stadionului.

3. PROIECTAREA ȘI ARHITECTURA SISTEMULUI

Etapă de proiectare reprezintă faza de tranziție critică de la analiza cerințelor și fundamentarea teoretică la implementarea efectivă a produsului software. În acest capitol, se va realiza modelarea arhitecturală a Dashboard-ului destinat administrării stadionului de mini-fotbal. Proiectarea sistemului implică definirea modului în care componentele interacționează, stabilirea schemelor de comunicare prin rețea și, cel mai important, modelarea structurii bazei de date. Un design arhitectural corect garantează că aplicația va fi scalabilă, securizată și capabilă să proceseze fără erori fluxurile zilnice de rezervări și tranzacții financiare.

3.1. Arhitectura aplicației și schemele de comunicare frontend-backend

Pentru a satisface cerințele complexe ale managementului sportiv și financiar, sistemul a fost proiectat pe baza unei arhitecturi stratificate (N-Tier Architecture), mai precis un model decuplat cu trei niveluri distincte (Three-Tier Architecture). Această separare structurală împarte responsabilitățile sistemului în Nivelul de Prezentare, Nivelul Logicii de Business și Nivelul de Stocare a Datelor.

1. Nivelul de Prezentare (frontend Layer) Acest nivel reprezintă interfața grafică cu care interacționează exclusiv administratorul stadionului. Dezvoltat integral în React.js, acest strat are rolul de a prelua datele brute de la server și de a le asambla în componente vizuale: tabele pentru evidența echipelor, calendare interactive pentru programarea orelor de joc și grafice analitice (prin React-Charts) pentru vizualizarea banilor din casă și a datoriilor. frontend-ul este proiectat să fie „reactiv” – adică să ofere un feedback vizual instantaneu la acțiunile utilizatorului, fără a reîncărca pagina, asigurând o navigare fluidă între secțiunile Dashboard-ului.

2. Nivelul Logicii de Business (backend / Application Layer) Reprezentând nucleul operațional al sistemului, acest nivel este construit pe platforma Node.js folosind framework-ul Express.js. Aici sunt găzduiți algoritmi și regulile de business ale afacerii. De exemplu, când administratorul asignează o oră de joc unei echipe, backend-ul execută un set de validări: verifică dacă intervalul orar nu este deja rezervat de o altă echipă, calculează costul total în funcție de tariful orar și actualizează balanța financiară (adăugând suma la restanțe dacă nu a fost achitată pe loc). Tot pe acest nivel funcționează și modulul de securitate, care validează identitatea administratorului pe baza unui token (JWT), blocând orice cerere neautorizată.

3. Nivelul de Stocare a Datelor (Data Layer) Baza de date reprezintă stratul fundațional, responsabil pentru persistența și integritatea pe termen lung a tuturor informațiilor introduse în

aplicație. Baza de date stochează entitățile esențiale și relațiile dintre ele: profilele angajaților, istoricul echipelor, programările efectuate, plățile încasate și cheltuielile de mentenanță ale stadionului.

Schema de comunicare și fluxul de date Interacțiunea dintre aceste trei niveluri se realizează printr-un API RESTful, care acționează ca un pod de comunicare. Datele sunt transmise exclusiv sub formă de obiecte JSON (JavaScript Object Notation). Pentru a ilustra schema de comunicare, putem analiza fluxul de date în cazul unei operațiuni zilnice obișnuite: *înregistrarea unei plăți pentru stingerea unei datorii*.

1. *Inițierea cererii (Clientul)*: Administratorul accesează secțiunea „Datorii Echipe” din Dashboard și introduce suma de 500 de lei achitată de o anumită echipă, apăsând butonul „Adaugă plată”. React captează acest eveniment și formulează o cerere HTTP de tip POST către o rută specifică a API-ului (ex: /api/finante/plata), atașând suma, ID-ul echipei și token-ul de securitate.
2. *Procesarea cererii (Serverul)*: Node.js primește cererea, validează token-ul de securitate, identifică echipa și preia logica matematică. Scade cei 500 de lei din totalul datoriilor echipei respective și adaugă aceeași sumă în parametrul „Bani în casă / Încasări totale”.
3. *Persistența (Baza de Date)*: Serverul trimite comenzile SQL sau NoSQL necesare pentru a salva definitiv aceste noi valori în baza de date.
4. *Răspunsul (Response)*: Odată ce datele au fost salvate cu succes, serverul trimite înapoi către frontend un cod de succes (Status 200 OK) și datele actualizate.
5. *Actualizarea vizuală (frontend)*: React primește răspunsul, își actualizează starea internă (State) și redesenează automat pe ecran graficul financiar și tabelul cu datorii, reflectând noua balanță.

Această arhitectură modulară asigură un flux informațional sigur, rapid și extrem de organizat, transformând aplicația într-un sistem robust, capabil să susțină managementul complet al unui obiectiv sportiv.

3.2. Proiectarea bazei de date și a schemelor conceptuale pentru entități

Proiectarea bazei de date este una dintre cele mai sensibile și importante etape în dezvoltarea unei aplicații web orientate spre gestiunea financiară și administrativă. O structură de date incorectă sau incompletă poate duce la redundanța informațiilor, pierderea trasabilității financiare și încetinirea generală a sistemului. Pentru Dashboard-ul dedicat stadionului de mini-fotbal, s-a optat pentru un

model de date bine structurat, care să garanteze integritatea calculelor matematice și relaționarea corectă între programări, echipe și încasări.

Arhitectura datelor a fost divizată logic în patru entități conceptuale majore, fiecare având o schemă clar definită, cu attribute specifice și relații interconectate.

1. Entitatea „Utilizatori” (Angajați / Administratori) Această schemă este responsabilă pentru managementul accesului și securitatea platformei. Întrucât sistemul gestionează date financiare sensibile (profitul net, banii din casă), accesul public trebuie interzis complet.

- *Attribute principale:* ID Unic, Nume de utilizator (Username), Rol (ex. Administrator Principal, Casier/Supraveghetor), Parolă (stocată exclusiv sub formă de hash criptografic, pentru a nu putea fi citită nici măcar în cazul compromiterii bazei de date) și Data ultimei autentificări.
- *Rolul în sistem:* Această entitate validează logarea în Dashboard și asigură emiterea token-urilor de securitate (JWT) necesare pentru fiecare interacțiune cu serverul.

2. Entitatea „Echipe” (Clienți) Acest tabel reprezintă nucleul de clienți ai stadionului. Majoritatea veniturilor unui teren de mini-fotbal provin de la echipe fidele, care joacă în mod recurent, săptămânal.

- *Attribute principale:* ID Echipă, Numele echipei (sau al căpitanului/persoanei de contact), Număr de telefon pentru notificări, Data înregistrării în sistem și, cel mai important atribut, **Balanța Financiară** (valoarea restanțelor/datoriilor curente).
- *Rolul în sistem:* Existența acestei entități simplifică drastic procesul de rezervare. Administratorul nu mai trebuie să introducă numele de la zero la fiecare oră jucată, ci doar selectează echipa dintr-o listă. Parametrul „Balanță Financiară” este actualizat automat de server ori de câte ori echipa acumulează o oră neachitată sau achită o datorie veche.

3. Entitatea „Rezervări” (Programări Orar) Această entitate gestionează managementul timpului și disponibilitatea fizică a terenului de mini-fotbal. Este tabelul cu cea mai mare frecvență de creare și actualizare a înregistrărilor.

- *Attribute principale:* ID Rezervare, Data calendaristică, Ora de început, Ora de sfârșit, Costul total al sesiunii, Statusul plății (Achitat pe loc / Amânat la plată) și o **Cheie Străină** (Foreign Key) care face legătura cu ID-ul Echipei din tabelul anterior.

- *Rolul în sistem:* Scopul principal este prevenirea suprapunerii orelor de joc. Când frontend-ul încearcă să adauge o programare, backend-ul interoghează rapid această entitate pentru a valida disponibilitatea intervalului orar. De asemenea, dacă statusul este setat pe „Amânat la plată”, sistemul adaugă automat costul sesiunii la datoria echipei respective.

4. Entitatea „Finanțe” (Încasări și Cheltuieli) Aceasta este entitatea vitală pentru generarea rapoartelor din Dashboard. Aici este vizibilă sănătatea financiară a afacerii. Toate fluxurile de numerar care intră și ies din casieria stadionului sunt stocate în această secțiune.

- *Atribute principale:* ID Tranzacție, Tipul tranzacției (Venit / Cheltuială), Categoria (ex. Chirie teren, Factură energie electrică, Salariu paznic, Mentenanță gazon sintetic), Suma (valoarea numerică absolută), Data tranzacției și Descrierea operațiunii.
- *Rolul în sistem:* Pe baza acestor înregistrări detaliate, serverul calculează balanța netă (banii reali din casă). Aceste date brute sunt extrase prin API, trimise către interfața React și injectate în componentele bibliotecii React-Charts. Rezultatul final este un set de diagrame grafice dinamice, care îi oferă administratorului o vizualizare imediată a profitabilității.

Interconectarea și Integritatea Datelor Modelul proiectat nu funcționează izolat. Structura a fost gândită astfel încât să se declanșeze actualizări automate între tabele. De exemplu, atunci când o echipă își achită o restanță acumulată anterior, aplicația nu realizează doar modificarea profilului echipei; ea scade datoria din tabelul „Echipe” și generează simultan o înregistrare pozitivă nouă în tabelul „Finanțe”. Această proiectare riguroasă elimină erorile umane de calcul și previne pierderile de capital, transformând baza de date în coloana vertebrală a platformei de management sportiv.

3.3. Proiectarea interfețelor utilizator (Wireframes și Design pentru Dashboard)

Ultima etapă a procesului de proiectare arhitecturală, înainte de implementarea efectivă a codului, o reprezintă designul interfeței utilizatorului (User Interface - UI) și definirea experienței de utilizare (User Experience - UX). În cazul aplicațiilor destinate managementului administrativ, cum este acest Dashboard pentru stadionul de mini-fotbal, accentul nu trebuie pus pe animații complexe sau elemente grafice încărcate, ci pe ergonomie, claritate și viteză de operare. Administratorul trebuie să poată găsi informația dorită (o programare, o datorie) în cel mai scurt timp și cu un număr minim de interacțiuni (click-uri).

Pentru a vizualiza structura aplicației, procesul a început cu realizarea unor schițe conceptuale (Wireframes). Acestea au definit un layout de tip „Admin Dashboard” standard, recunoscut pentru

eficiența sa, fiind împărțit în trei zone vizuale principale: Meniul Lateral (Sidebar), Bara Superioară (Header) și Zona Principală de Conținut (Main Workspace).

1. Meniul Lateral (Sidebar-ul de navigare) Meniul lateral acționează ca panou principal de comandă și este vizibil permanent în partea stângă a ecranului. Acesta conține link-uri rapide către modulele aplicației, structurate logic:

- *Acasă (Panou Central):* Locul unde administratorul vede rezumatul zilei.
- *Calendar / Programări:* Secțiunea dedicată gestiunii timpului.
- *Echipe:* Baza de date cu profilurile clienților fideli.
- *Finanțe:* Modulul detaliat pentru introducerea facturilor și a încasărilor.
- *Datorii:* O secțiune izolată, evidențiată vizual, pentru a urmări rapid echipele cu restanțe.

2. Zona Principală de Conținut și Ecranele Cheie Aceasta este aria dinamică a aplicației, unde componenta React randează informațiile specifice secțiunii selectate din meniul. Proiectarea celor mai importante ecrane a urmat principii stricte de organizare a datelor:

- *Ecranul Panou Central (Dashboard-ul Principal):* Este ecranul de aterizare după autentificare. În partea superioară, interfața utilizează „Carduri KPI” (Key Performance Indicators) – casete vizuale colorate distinct (ex. Verde pentru „Bani în casă”, Roșu pentru „Total Datorii curente”, Albastru pentru „Total Echipe Active”). Sub aceste carduri se află zona analitică: graficele generate de librăria React-Charts. Aici, administratorul poate vedea o diagramă cu bare care compară încasările cu cheltuielile pe luna în curs, oferind o perspectivă clară asupra rentabilității stadionului fără a citi rapoarte detaliate.
- *Ecranul Calendarului de Programări:* Designul acestui ecran a fost gândit sub forma unui tabel orar. Orele libere au un fundal neutru (alb/gri), în timp ce orele ocupate sunt marcate prin blocuri colorate, conținând numele echipei rezervate. Printr-un simplu click pe un interval liber, se deschide un formular modal care permite selectarea rapidă a echipei și salvarea programării, optimizând astfel timpul de lucru.
- *Ecranul Echipe și Datorii:* Informația este prezentată sub formă de tabele interactive (DataGrids). Pentru o experiență de utilizare optimă, tabelele sunt dotate cu funcții de filtrare și căutare în timp real. Astfel, dacă administratorul caută o anumită echipă, introducerea primelor litere va filtra lista instantaneu. În dreptul fiecărei echipe cu restanțe, interfața integrează un buton clar de acțiune („Adaugă Plată”), care deschide un mic formular pentru a stinge datoria respectivă.

3. Ergonomie și Design Adaptabil (Responsive Web Design) O cerință non-funcțională critică stabilită în capitolul anterior este accesibilitatea sistemului de pe diferite dispozitive. Designul interfeței a fost proiectat cu o abordare flexibilă, folosind grile (CSS Grid/Flexbox). Dacă aplicația este accesată de pe un telefon mobil – de exemplu, când administratorul nu se află fizic la stadion, dar trebuie să verifice o rezervare – interfața se adaptează automat. Meniul lateral se ascunde în spatele unui buton de tip „hamburger”, cardurile financiare se aliniază pe verticală, iar tabelele devin scrollabile, garantând astfel o funcționalitate completă indiferent de rezoluția ecranului.

În concluzie, proiectarea arhitecturii serverului, a bazei de date și a interfeței utilizatorului creează fundamentul complet al aplicației. Acest design decuplat și orientat către utilizator permite trecerea la următoarea etapă a proiectului: implementarea practică și scrierea codului sursă.

4. IMPLEMENTAREA PRACTICĂ A PLATFORMEI

Acest capitol descrie procesul de materializare a conceptelor teoretice și arhitecturale stabilite anterior într-un produs software complet funcțional. Implementarea practică presupune scrierea efectivă a codului sursă, configurarea mediului de dezvoltare, integrarea bibliotecilor externe și conectarea interfeței vizuale cu logica de server. Etapele descrise în continuare reflectă modul în care cerințele de business ale stadionului de mini-fotbal au fost traduse în algoritmi și componente tehnice, rezultând un Dashboard capabil să proceseze autonom rezervările și calculele financiare.

4.1. Dezvoltarea serverului backend și configurarea API-ului

Prima etapă în dezvoltarea aplicației a constat în construirea infrastructurii de server (backend), care acționează ca motorul principal de procesare a datelor. Pentru această componentă s-a utilizat mediul de execuție Node.js, împreună cu framework-ul Express.js, recunoscut pentru eficiența sa în crearea de API-uri RESTful robuste și scalabile.

Inițializarea proiectului și securitatea rețelei Procesul a debutat cu inițializarea unui nou proiect Node.js (prin comanda `npm init`) și instalarea modulelor esențiale. Un aspect critic abordat încă din primele linii de cod a fost securitatea comunicațiilor. În acest sens, a fost configurat modulul CORS (Cross-Origin Resource Sharing) pentru a permite exclusiv interfeței frontend (găzduită pe un anumit domeniu, cum ar fi Vercel) să comunice cu serverul. Totodată, variabilele sensibile, cum ar fi adresa bazei de date și cheile secrete pentru criptare, au fost izolate în fișiere de mediu (`.env`), prevenind expunerea acestora în codul sursă public.

Implementarea schemelor bazei de date Conform arhitecturii stabilite în capitolul 3, au fost create modelele de date (Models) pentru entitățile principale: Echipe, Rezervări și Finanțe. Utilizând un sistem de mapare a obiectelor (ORM/ODM), tabelele au fost traduse în scheme de cod JavaScript. De exemplu, schema pentru „Echipă” a fost instanțiată pentru a accepta obligatoriu un nume (de tip *String*) și un parametru implicit pentru datoria curentă (de tip *Number*, setat inițial la valoarea 0).

Dezvoltarea rutelor (Endpoints) și a logicii de business Nucleul funcționalității backend-ului este reprezentat de controlere (Controllers), funcțiile care dictează ce se întâmplă atunci când administratorul interacționează cu interfața. Pentru a gestiona stadionul, au fost dezvoltate și expuse mai multe rute esențiale:

- **Rutele pentru Echipe (/api/teams):** A fost creată o rută de tip GET pentru a extrage din baza de date lista completă a echipelor și datoriile acestora. Această rută este apelată automat de

frontend la deschiderea Dashboard-ului. O altă rută de tip POST permite înregistrarea unei echipe noi direct din interfață.

- **Rutele pentru Programări (/api/reservations):** Logica implementată aici este mai complexă. La primirea unei cereri de tip POST pentru o nouă rezervare, funcția de control verifică mai întâi în baza de date dacă intervalul orar dorit este disponibil la data specificată. Dacă este ocupat, serverul blochează operațiunea și returnează o eroare (Status 400). Dacă terenul este liber, programarea este salvată, iar sistemul declanșează automat actualizarea datelor financiare.
- **Rutele Financiare și de Datorii (/api/finances):** Aceasta este secțiunea care automatizează calculele manuale ale administratorului. A fost creat un endpoint specific pentru înregistrarea plăților. Când administratorul apasă butonul „Adaugă plată” pentru o echipă care are o restanță, serverul preia suma, execută o operațiune matematică de scădere asupra atributului „Datorie curentă” al echipei și, simultan, creează o nouă înregistrare în tabelul de „Încasări”, adăugând suma la profitul net („banii din casă”).

Această decuplare a calculelor matematice (mutarea lor pe server) asigură că, indiferent de performanța dispozitivului de pe care este accesat Dashboard-ul (telefon sau laptop), datele financiare vor fi întotdeauna calculate instantaneu, precis și fără riscul unor erori de rețea care să corupă balanța afacerii.

4.2. Construirea interfeței web (~~frontend~~) și integrarea graficelor

Odată cu finalizarea arhitecturii de server (backend) și expunerea API-ului RESTful, faza următoare a constat în dezvoltarea și adaptarea interfeței grafice (frontend). Punctul de plecare pentru această componentă l-a constituit proiectul dezvoltat inițial în cadrul tezei de an, care a necesitat o refactorizare majoră. Obiectivul a fost transformarea unei interfețe cu date statice (mock-data) într-o aplicație web dinamică, capabilă să comunice în timp real cu baza de date, devenind astfel instrumentul principal de lucru pentru administratorul stadionului. Pentru a asigura o accesibilitate globală, aplicația frontend a fost compilată și implementată (deployed) pe platforma de cloud hosting Vercel.

Gestionarea stării și apelurile asincrone (Integrarea API-ului) La nivelul codului frontend, dezvoltat în React.js, inima aplicației o reprezintă comunicarea cu serverul Node.js. Navigarea între secțiunile Dashboard-ului (Calendar, Echipe, Finanțe) este fluidă, fiind gestionată de biblioteca react-router-dom fără a reîncărca pagina.

Provocarea tehnică principală a fost sincronizarea stării aplicației (State Management) cu baza de date. În acest sens, s-au utilizat extensiv React Hooks, în special `useEffect` și `useState`. De exemplu, în momentul în care administratorul accesează secțiunea „Echipe și Datorii”, hook-ul `useEffect` declanșează imediat o cerere asincronă (folosind `fetch` sau `axios`) către ruta `/api/teams` a serverului. Serverul returnează un fișier JSON conținând lista tuturor echipelor și balanța lor financiară curentă. frontend-ul preia acest răspuns, îl stochează în funcția `useState` și randează automat tabelul interactiv pe ecran.

Pentru operațiunile de scriere, fluxul este inversat. Dacă o echipă achită o restanță, administratorul completează un scurt formular în interfață. La apăsarea butonului de confirmare, React colectează valoarea, formulează o cerere HTTP de tip POST și o expediază către server, acompaniată de token-ul de securitate (JWT). Imediat ce serverul confirmă actualizarea bazei de date (Status 200 OK), interfața se re-randează pentru a reflecta noua valoare a datoriilor și a banilor din casă, oferind un feedback vizual instantaneu.

Integrarea React-Charts pentru analiza datelor financiare O componentă definitorie a acestui Dashboard, care aduce o plusvaloare majoră în administrarea afacerii, este modulul de vizualizare analitică a datelor. Simplul afișaj tabelar al încasărilor și cheltuielilor nu este suficient pentru a evalua rapid rentabilitatea stadionului. Prin urmare, s-a integrat biblioteca avansată **React-Charts** (precum și extensii de tip geo-charts pentru analize complexe).

Implementarea graficelor presupune un proces de transformare a datelor. Datele brute primite de la server (ex: un șir de 50 de tranzacții individuale efectuate în cursul unei luni) trebuie formate specific înainte de a fi injectate în componenta `<Chart>`. Algoritmii din frontend agregă aceste valori: adună toate încasările din chiria terenului și scad toate cheltuielile operaționale (curent, salarii, curățenie), generând valoarea netă a „banilor din casă”.

Aceste date agregate alimentează mai multe reprezentări vizuale esențiale plasate pe ecranul principal (Panoul de Control):

1. **Diagrama evoluției financiare (Bar Chart / Line Chart):** Un grafic care pune în antiteză, pe o axă a timpului, coloana încasărilor zilnice cu cea a cheltuielilor. Acest grafic îi permite administratorului să identifice vizual zilele de vârf și să estimeze profitabilitatea săptămânală.
2. **Graficul restanțelor (Pie Chart / Doughnut Chart):** O reprezentare vizuală a distribuției veniturilor, care arată clar ponderea banilor deja încasați față de ponderea datoriilor pe care

echipele trebuie să le achite. Dacă segmentul datoriilor crește disproporționat, administratorul știe vizual că trebuie să intervină.

Prin această integrare strânsă între React, comunicarea asincronă prin API și redarea vizuală prin React-Charts, aplicația transcende stadiul de simplu tabel digital. Ea devine un instrument analitic inteligent, stabil și extrem de performant, capabil să optimizeze semnificativ timpul și precizia managementului sportiv.

5. TESTAREA SISTEMULUI ȘI MANUALUL DE UTILIZARE

Introducere la Capitolul 5 Dezvoltarea unui produs software nu se încheie odată cu scrierea ultimelor linii de cod. Pentru ca o aplicație să fie pregătită pentru lansarea în mediul de producție și utilizarea zilnică, este obligatorie validarea acesteia printr-un proces riguros de testare. Întrucât Dashboard-ul este destinat administrării financiare și operaționale a unui stadion de mini-fotbal, orice eroare de calcul sau de suprapunere a rezervărilor poate genera pierderi de capital sau nemulțumiri din partea clienților. Acest capitol descrie scenariile de testare aplicate pentru validarea logicii de business și prezintă ghidul de utilizare (manualul) conceput pentru a facilita interacțiunea administratorului cu platforma.

5.1. Testarea funcționalităților (backend și frontend)

Pentru a garanta un nivel înalt de calitate (Quality Assurance), procesul de testare a fost împărțit în două direcții principale: testarea la nivel de server (API/backend) și testarea la nivel de interfață (UI/frontend).

Testarea API-ului și a logicii backend-ului Logica matematică a aplicației a fost testată pentru a confirma că rutele procesează corect cererile HTTP. S-au utilizat instrumente dedicate (precum Postman) pentru a simula diverse scenarii de interacțiune directă cu baza de date, ocolind interfața grafică. Scenariile critice testate au inclus:

1. *Scenariul de validare a programărilor:* S-a expediat o cerere de tip POST pentru a rezerva terenul de mini-fotbal între orele 18:00 și 19:00 la o anumită dată. Ulterior, s-a expediat o a doua cerere identică. Sistemul a reacționat conform cerințelor, respingând a doua cerere și returnând un cod de eroare (400 Bad Request), prevenind astfel suprapunerea orelor de joc.
2. *Scenariul integrității financiare:* S-a testat funcția de „Adăugare Plată” pentru o echipă care figura cu o datorie curentă de 1000 de lei. S-a simulat introducerea unei plăți parțiale de 400 de lei. Testul a confirmat că serverul a redus datoria echipei la 600 de lei și, simultan, a adăugat automat suma de 400 de lei în parametrul general al „banilor din casă”.

Testarea Interfeței Utilizatorului (frontend) și UAT La nivelul clientului (React), testarea s-a concentrat pe funcționalitate, ergonomie și responsivitate. S-a realizat testarea de tip *User Acceptance Testing (UAT)*, simulând acțiunile reale ale unui administrator grăbit.

- *Validarea formularelor:* S-a testat comportamentul formularelor la introducerea unor date eronate (de exemplu, introducerea de litere în câmpul destinat sumelor de bani). Interfața a

reacționat corect, afișând avertismente vizuale (mesaje cu roșu) și blocând trimiterea datelor către server până la corectarea acestora.

- *Testarea graficelor (React-Charts):* S-a verificat dacă diagramele se actualizează dinamic imediat după înregistrarea unei tranzacții financiare noi, confirmând utilitatea lor de analiză în timp real.
- *Responsivitatea:* Aplicația a fost deschisă pe diverse dispozitive (laptop, tabletă și ecran de telefon mobil). Grilele CSS s-au adaptat corect, meniul a devenit de tip *hamburger* pe mobile, iar tabelele financiare au rămas lizibile, garantând accesibilitatea de oriunde.

5.2. Ghid de utilizare a platformei pentru administrator

Pentru o tranziție ușoară de la evidența clasică (agendă și pix) la soluția digitală, a fost elaborat un set clar de instrucțiuni de utilizare a platformei, axat pe fluxul zilnic de lucru.

1. **Autentificarea în sistem:** Accesați link-ul platformei (găzduite pe Vercel). Pe ecranul principal, introduceți numele de utilizator (ex. *admin_stadion*) și parola securizată. Odată validat, veți fi redirecționat automat către Panoul Central (Dashboard).
2. **Consultarea situației financiare (Panoul Central):** Aici puteți vizualiza instantaneu starea afacerii. Cardurile superioare indică „Banii din casă” și „Datoria totală de recuperat”. Graficele ilustrează ponderea încasărilor față de cheltuielile lunii curente.
3. **Adăugarea și gestionarea echipelor:** Navigați în meniul lateral la secțiunea „Echipe”. Pentru a înregistra un client nou, apăsați butonul „Adaugă Echipă Nouă”, introduceți numele echipei și numărul de telefon al căpitanului. Dacă echipa acumulează datorii, acestea vor fi afișate automat în coloana „Balanță Financiară”.
4. **Efectuarea unei rezervări de teren:** Accesați modulul „Calendar/Programări”. Faceți click pe un interval orar liber din ziua dorită. Se va deschide o fereastră modală: selectați echipa din lista derulantă și bifați dacă ora este „Achitată pe loc” sau „Amânată la plată” (caz în care suma se va adăuga la datoria echipei). Apăsați „Salvează programarea”.
5. **Gestionarea banilor și a datoriilor:** În secțiunea „Finanțe și Datorii”, veți găsi evidența clară a banilor. Pentru a stinge restanța unei echipe, căutați echipa respectivă în tabel, apăsați pe butonul de acțiune „Plată Nouă”, introduceți suma primită de la căpitan și salvați. Sistemul va actualiza automat datoria echipei și va crește profitul net indicat în Panoul Central. Prin adăugarea facturilor (ex. energie electrică) la secțiunea Cheltuieli, platforma va recalcula instantaneu suma reală disponibilă în casierie.

CONCLUZII

Elaborarea prezentei teze de licență a pornit de la o necesitate reală și acută din mediul antreprenorial modern: nevoia de a digitaliza și automatiza procesele de management operațional și financiar. Obiectivul central al acestei lucrări a constat în proiectarea, implementarea și testarea unei aplicații web integrate, de tip Dashboard Full-Stack, destinată eficientizării administrării unui stadion de mini-fotbal. Odată cu finalizarea etapelor de dezvoltare, putem afirma că acest obiectiv a fost atins cu succes, rezultatul fiind un produs software viabil, securizat și perfect adaptat cerințelor de business ale beneficiarului.

Analiza aprofundată a domeniului, realizată în prima parte a lucrării, a evidențiat vulnerabilitățile metodelor tradiționale de gestiune. Utilizarea suportului de hârtie sau a foilor de calcul nesincronizate pentru evidența orelor de joc, a cheltuielilor și a datoriilor acumulate de echipe s-a dovedit a fi o abordare inefficientă, generatoare de erori umane și consumatoare de timp. Răspunsul tehnologic la aceste provocări l-a constituit crearea unui sistem informatic personalizat, bazat pe o arhitectură Client-Server decuplată, care mută greutatea calculelor și a stocării datelor în mediul digital.

Din perspectivă tehnologică, lucrarea a demonstrat capacitatea de integrare a unui ecosistem modern de dezvoltare bazat pe JavaScript. Pe nivelul de prezentare (frontend), utilizarea bibliotecii **React.js** a permis crearea unei interfețe modulare, fluide și intuitive (Single Page Application). Prin integrarea librăriei **React-Charts**, aplicația transcende funcția de simplu registru digital, devenind un instrument analitic puternic. Datele financiare brute sunt transformate instantaneu în reprezentări grafice dinamice, oferind administratorului o viziune clară, în timp real, asupra balanței dintre încasări, cheltuieli operaționale și soldul debitor (banii din casă comparativ cu restanțele de recuperat).

Pilonul central care conferă stabilitate și securitate aplicației este nivelul de server (backend), dezvoltat prin intermediul **Node.js** și **Express.js**. Această arhitectură a facilitat construirea unui API RESTful robust, capabil să proceseze asincron cererile interfeței. Crearea unei baze de date structurate a rezolvat definitiv problema integrității informaționale: algoritmi de server validează automat disponibilitatea orelor pentru a preveni suprapunerea rezervărilor și recalculează automat datoria unei echipe în momentul înregistrării unei plăți. În plus, implementarea mecanismului de autentificare bazat pe token-uri (JWT) a asigurat faptul că datele financiare sensibile ale stadionului sunt strict protejate împotriva accesului neautorizat.

Semnificația practică a acestei teze este incontestabilă. Produsul software rezultat nu este un simplu prototip academic, ci o soluție operațională imediat aplicabilă. Implementarea acestui Dashboard în activitatea curentă a stadionului de mini-fotbal va reduce drastic timpul alocat sarcinilor

birocratice, va elimina erorile de calcul contabil și va optimiza fluxul de numerar prin evidența strictă a creanțelor (datoriilor echipelor).

Perspective de dezvoltare ulterioară Deși platforma în stadiul actual acoperă toate cerințele fundamentale ale administrării bazei sportive, arhitectura sa scalabilă permite extinderi funcționale majore în viitor. Pentru dezvoltarea ulterioară a proiectului, se propun următoarele direcții de modernizare:

- **Integrarea unui modul de notificări automate:** Conectarea aplicației la un serviciu extern (ex. Twilio sau SendGrid) pentru a trimite automat mesaje SMS sau e-mail-uri echipelor cu o zi înainte de rezervare sau pentru a le reaminti de scadența datoriilor.
- **Dezvoltarea unei interfețe (aplicații) destinate clienților:** Crearea unui modul separat (Customer Portal) unde căpitaniii echipelor să se poată autentifica pentru a-și vedea propriul istoric de meciuri, datoriile curente și pentru a solicita o rezervare direct online, care să fie ulterior doar aprobată de administrator.
- **Integrarea unui modul de plăți online:** Conectarea Dashboard-ului cu un procesator de plăți (ex. Stripe), permițând echipelor să își achite datoriile direct cu cardul bancar, automatizând astfel complet fluxul de încasări.

În concluzie, lucrarea confirmă faptul că tranziția către managementul digitalizat este esențială pentru succesul oricărei afaceri moderne, iar tehnologiile web actuale oferă instrumentele necesare pentru a construi sisteme sigure, analitice și perfect adaptate la logica specifică a fiecărui model de business.

BIBLIOGRAFIE

1. WIERUCH, R. The Road to React: Your journey to master plain yet pragmatic React.js. Independently published, 2020. ISBN 9781720043997.
2. EISENMAN, B. Learning React: A Hands-On Guide to Building Web Applications Using React and Redux. Addison-Wesley, 2017. ISBN 9780134843551.
3. LINDLEY, J. Mastering JavaScript Functional Programming. Packt Publishing, 2016. ISBN 9781785882166.
4. CHAN, E. P., BANKS, A. Learning React: Modern Patterns for Developing React Apps. O'Reilly Media, 2023. ISBN 9781098113612.
5. React Documentation – React Official Site. [online] [citat 10.02.2025]. Disponibil: <https://react.dev>
6. TanStack Charts Documentation – React-Charts. [online] [citat 15.02.2025]. Disponibil: <http://react-charts.tanstack.com/docs/api>
7. FreeCodeCamp – React Curriculum. [online] [citat 22.02.2025]. Disponibil: <https://www.freecodecamp.org/learn>
8. Codecademy – Learn React. [online] [citat 01.03.2025]. Disponibil: <https://www.codecademy.com/learn/react-101>
9. MDN Web Docs – JavaScript, HTML, CSS. [online] [citat 05.03.2025]. Disponibil: <https://developer.mozilla.org>
10. Pluralsight – React Fundamentals. [online] [citat 12.03.2025]. Disponibil: <https://www.pluralsight.com/courses/react-fundamentals>
11. Udemy – Modern React with Redux. [online] [citat 18.03.2025]. Disponibil: <https://www.udemy.com/course/react-redux/>
12. TanStack – React Charts GitHub Repo. [online] [citat 25.03.2025]. Disponibil: <https://github.com/TanStack/react-charts>
13. JSX Introduction – W3Schools. [online] [citat 02.04.2025]. Disponibil: https://www.w3schools.com/react/react_jsx.asp
14. React TypeScript Cheatsheet. [online] [citat 15.04.2025]. Disponibil: <https://react-typescript-cheatsheet.netlify.app/>
15. frontend Masters – Advanced React Patterns. [online] [citat 05.05.2025]. Disponibil: <https://frontendmasters.com/courses/advanced-react-patterns/>